Лабораторийн ажил №8

Үндсэн санах ой (Main Memory) — Бүлэг 9, Practice Exercises

F.CSM302 Үйлдлийн систем
Гүйцэтгэсэн: Б.Билгүүн (B190910014)

Сурах бичиг: A. Silberschatz, P. B. Galvin, G. Gagne, Operating System Concepts, 10th ed., Wiley, 2018

Хуудаслалтад (paging) логик хаяг нь **хуудасны дугаар** p ба **оффсет** d хоёр хэсэгтэй: хуудасны хэмжээ 2^n байт бол $p = \lfloor \text{хаяг} / 2^n \rfloor$ (дээд битүүд), $d = \text{хаяг} \bmod 2^n$ (доод n бит). Хуудасны хүснэгт p-г фрэймийн дугаар f руу буулгаж, физик хаяг $= f \cdot 2^n + d$ болно.

Дасгал 9.1

Асуулт. Логик ба физик хаягийн хоёр ялгааг нэрлэ.

Хариулт.

1. **Хэн үүсгэх, хэн харах.** Логик (виртуал) хаягийг CPU программ ажиллах үед үүсгэдэг; хэрэглэгчийн программ зөвхөн логик хаягтай харьцаж, бодит физик хаягийг хэзээ ч хардаггүй. Физик хаяг бол санах ойн нэгж (memory unit)-д бодитоор очиж, санах ойн хаягийн регистрт ачаалагдах хаяг юм.
2. **Хаягийн орон зай ба хөрвүүлэлт.** Логик хаяг нь процесс бүрийн өөрийн хаягийн орон зайд (ихэвчлэн 0-ээс эхэлдэг, бусад процессынхоос тусдаа) байна; физик хаяг нь бүх процесст нэгдсэн бодит санах ойн хаяг. Execution-time холболтод логик хаягийг MMU (шилжилтийн регистр, хуудасны хүснэгт)-ээр физик хаяг руу ажиллах явцад хөрвүүлдэг тул хоёулаа ялгаатай; compile/load-time холболтод ижил байна.

Дасгал 9.2

Асуулт. Хуудасны хэмжээ яагаад үргэлж 2-ын зэрэг байдаг вэ?

Хариулт.

Хуудасны хэмжээ 2^n байвал логик хаягийг хуудасны дугаар ба оффсет болгон хуваах нь зөвхөн битүүдийг таслах үйлдэл болно: доод n бит — оффсет, үлдсэн дээд битүүд — хуудасны дугаар. Хуваах, үлдэгдэл олох үйлдлийг шилжүүлэх (>>) ба маск (&) үйлдлээр, эсвэл техник хангамжид зүгээр л утсаар салгаж хийнэ — ямар нэг арифметикгүй, хурдан. Мөн физик хаяг $=$ фрэйм $\cdot 2^n +$ оффсет-ийг фрэймийн дугаарыг оффсеттэй залгах (concatenate)-аар гаргана. 2-ын зэрэг биш байвал хаяг хөрвүүлэх

бүрд жинхэнэ хуваах үйлдэл шаардагдаж, санах ойн хандалт бүрийг удаашруулна. (Энэ санааг `lab8/addresses.c` хэрэгжүүлдэг.)

Дасгал 9.3

Асуулт. Программыг код ба өгөгдөл гэсэн хоёр хэсэгт хуваадаг систем. CPU заавар уншиж байна уу, өгөгдөлд хандаж байна уу гэдгийг мэддэг тул заавар ба өгөгдөлд тус бүр base–limit регистрийн хос өгөгдсөн. Зааврын хос автоматаар зөвхөн уншигдах тул программыг олон хэрэглэгч хуваалцаж болно. Давуу ба сул талыг хэлэлц.

Хариулт.

Давуу тал:

- **Код хуваалцах:** нэг программын (жнь текст засварлагч) кодыг санах ойд нэг л хувь байлгаж, олон хэрэглэгч тус тусын өгөгдлийн хэсэгтэйгээр ашиглана — санах ой хэмнэнэ.
- **Хамгаалалт:** кодын хэсэг зөвхөн уншигдах тул программ өөрийн кодыг санамсаргүй/хорлонтойгоор өөрчлөх боломжгүй; өгөгдөл рүү хандалт өгөгдлийн хос дотор шалгагдана.
- Код ба өгөгдлийг санах ойд тусад нь, үргэлжилсэн байх албагүй хоёр хэсэгт байрлуулах тул байршуулахад илүү уян хатан, гадаад хоосон зай (fragmentation) бага.

Сул тал:

- Нэмэлт регистрийн хос, хандалт бүрт аль хосыг ашиглахыг шийдэх логик — техник хангамж ба контекст сэлгэлтийн ачаалал (хадгалах/сэргээх регистр олширно).
- Зөвхөн хоёр хэсэг: стек, heap, хуваалцсан сан зэргийг тус тусад нь хамгаалах/хуваалцах боломжгүй — сегментчлэл, хуудаслалтаас хамаагүй бүдүүлэг.
- Код ба өгөгдөл тус бүр үргэлжилсэн байх ёстой тул гадаад хоосон зай үүсэх асуудал хэвээр; өөрийгөө өөрчилдөг код (self-modifying code) ажиллахгүй.
- Хуваалцсан код нь бүх хэрэглэгчид ижил логик хаягт байх, эсвэл байршилаас үл хамаарах (position-independent) байх шаардлагатай.

Дасгал 9.4

Асуулт. 1024 үгтэй 64 хуудаснаас бүрдэх логик хаягийн орон зай 32 фрэймтэй физик санах ойд буулгагдана. (a) Логик хаяг хэдэн бит вэ? (b) Физик хаяг хэдэн бит вэ?

Хариулт.

Оффсет: $1024 = 2^{10}$ → 10 бит.

(a) Хуудас: $64 = 2^6$ → 6 бит. Логик хаяг $= 6 + 10 =$ **16** бит.

(b) Фрэйм: $32 = 2^5$ → 5 бит. Физик хаяг $= 5 + 10 =$ **15** бит.

Дасгал 9.5

Асуулт. Хуудасны хүснэгтийн хоёр бичлэг санах ойн нэг фрэймийг заавал ямар үр дагавар гарах вэ? Үүнийг их хэмжээний санах ойг нэг газраас нөгөө рүү хуулах хугацааг багасгахад хэрхэн ашиглаж болох вэ? Нэг хуудсанд байт өөрчлөхөд нөгөө хуудсанд ямар нөлөө гарах вэ?

Хариулт.

Үр дагавар: хоёр өөр логик хуудас (нэг процесст эсвэл хоёр өөр процесст) нэг физик фрэймд буулгагдаж, ижил өгөгдлийг хуваалцана — хуваалцсан санах ой (shared pages) гэсэн үг.

Хурдан хуулах: өгөгдлийг байт байтаар хуулахын оронд хүрэх хуудасны хүснэгтийн бичлэгүүдийг эх хуудсуудын фрэйм рүү заалгана — хэдэн мянган байтыг хуулахын оронд хэдэн бичлэг шинэчлэх л үйлдэл (жнь `fork()`-д бүх хаягийн орон зайг хуулахгүй байх).

Байт өөрчлөх нөлөө: хоёр бичлэг ижил фрэймийг зааж байгаа тул нэгээр нь өөрчлөхөд нөгөөд нь мөн харагдана — хуулбар биш, нэг л өгөгдөл. Тэгэхээр хуулбар бие даасан байх ёстой бол хуудсыг зөвхөн уншигдах гэж тэмдэглэж, бичих оролдлого гарах үед л тухайн хуудсыг жинхэнээр хувилж тусдаа фрэйм өгнө — **copy-on-write** (§10.3). Ингэснээр өөрчлөгдөөгүй хуудсууд хуваалцсан хэвээр, хуулалт зөвхөн шаардлагатай үед хийгдэнэ.

Дасгал 9.6

Асуулт. 300 KB, 600 KB, 350 KB, 200 KB, 750 KB, 125 KB гэсэн зургаан хуваалт (дарааллаар) өгөгдсөн. First-fit, best-fit, worst-fit алгоритмууд 115 KB, 500 KB, 358 KB, 200 KB, 375 KB процессуудыг (дарааллаар) хэрхэн байрлуулах вэ?

Хариулт.

Хувьсах хуваалтын (variable partition) схем: сонгосон чөлөөт хуваалт (hole)-аас процессыг өгч, үлдэгдэл нь шинэ жижиг нүх болж үлдэнэ.

Процесс	First-fit	Best-fit	Worst-fit
115 KB	300 KB (үлд. 185)	125 KB (үлд. 10)	750 KB (үлд. 635)
500 KB	600 KB (үлд. 100)	600 KB (үлд. 100)	635 KB (үлд. 135)
358 KB	750 KB (үлд. 392)	750 KB (үлд. 392)	600 KB (үлд. 242)
200 KB	350 KB (үлд. 150)	200 KB (үлд. 0)	350 KB (үлд. 150)
375 KB	392 KB (үлд. 17)	392 KB (үлд. 17)	**багтахгүй** — хүлээнэ

Worst-fit-ийн хувьд 375 KB ирэхэд хамгийн том нүх 300 KB тул багтахгүй — нийт чөлөөт зай хангалттай ч гадаад хоосон зай (external fragmentation)-ын улмаас процесс хүлээнэ. Энэ жишээнд first-fit, best-fit хоёулаа бүх процессыг байрлуулсан; best-fit бүтэн 200 KB нүхийг яг тааруулж ашигласан. (`lab8/memcalc.py`-ээр шалгасан.)

Дасгал 9.7

Асуулт. 1 KB хуудастай гэж үзээд дараах (аравтын) хаягуудын хуудасны дугаар ба оффсетийг ол: 3085, 42095, 215201, 650000, 2000001.

Хариулт.

$1\ \text{KB} = 1024 = 2^{10}$: хуудас $= \lfloor a/1024 \rfloor$, оффсет $= a \bmod 1024$.

Хаяг	Хуудас	Оффсет	Шалгалт
3085	3	13	$3 \cdot 1024 + 13$
42095	41	111	$41 \cdot 1024 + 111$
215201	210	161	$210 \cdot 1024 + 161$
650000	634	784	$634 \cdot 1024 + 784$
2000001	1953	129	$1953 \cdot 1024 + 129$

Дасгал 9.8

Асуулт. BTV үйлдлийн систем 21 бит виртуал хаягтай, зарим төхөөрөмж дээр ердөө 16 бит физик хаягтай; хуудасны хэмжээ 2 KB. (a) Энгийн нэг түвшний хуудасны хүснэгт, (b) урвуу хуудасны хүснэгт тус бүр хэдэн бичлэгтэй вэ? Хамгийн их физик санах ой хэд вэ?

Хариулт.

Хуудас $2\ \text{KB} = 2^{11}$ → оффсет 11 бит.

(a) Виртуал хуудасны дугаар $21 - 11 = 10$ бит → $2^{10} =$ **1024** бичлэг (виртуал хуудас бүрт нэг).

(b) Урвуу хүснэгт физик фрэйм бүрт нэг бичлэгтэй: фрэймийн дугаар $16 - 11 = 5$ бит → $2^5 = \mathbf{32}$ бичлэг.

Хамгийн их физик санах ой: 2^{16} байт = **64 KB** (32×2 KB).

Дасгал 9.9

Асуулт. 4 KB хэмжээтэй 256 хуудастай логик хаягийн орон зай 64 фрэймтэй физик санах ойд буулгагдана. (a) Логик хаяг хэдэн бит? (b) Физик хаяг хэдэн бит?

Хариулт.

Оффсет: 4 KB $= 2^{12}$ → 12 бит.

(a) Хуудас $256 = 2^8$ → 8 бит; логик хаяг $8 + 12 = \mathbf{20}$ бит.

(b) Фрэйм $64 = 2^6$ → 6 бит; физик хаяг $6 + 12 = \mathbf{18}$ бит.

Дасгал 9.10

Асуулт. 32 бит логик хаяг, 4 KB хуудастай систем 512 MB хүртэл физик санах ойг дэмждэг. (a) Энгийн нэг түвшний хуудасны хүснэгт, (b) урвуу хуудасны хүснэгт хэдэн бичлэгтэй вэ?

Хариулт.

(a) Хуудасны дугаар $32 - 12 = 20$ бит → $2^{20} = \mathbf{1\,048\,576}$ бичлэг (процесс бүрт). Бичлэг 4 байт бол хүснэгт дангаараа 4 MB — иймээс олон түвшний/хэшилсэн хүснэгт хэрэглэдэг.

(b) Фрэймийн тоо $= 512$ MB$/4$ KB $= 2^{29}/2^{12} = 2^{17} = \mathbf{131\,072}$ бичлэг (системд нэг хүснэгт).

Programming Problem 9.28 (бичлэгт)

32 бит виртуал хаяг, 4 KB хуудас: `lab8/addresses.c` аравтын хаягийг командын мөрөөс авч `uint32_t`-д хадгалаад хуудасны дугаар = `address >> 12`, оффсет = `address & 0xFFF`-ийг хэвлэнэ. Сөрөг, тоо биш, $2^{32}-1$-ээс их оролтыг алдаатай гэж мэдэгдэнэ.

```
$ ./addresses 19986
The address 19986 contains:
page number = 4
offset = 3602
$ ./addresses 4294967295        # 2^32 - 1
page number = 1048575, offset = 4095
```

```
$ ./addresses 4294967296
Error: 4294967296 does not fit in a 32-bit address (max 4294967295)
```